

Optimization Dynamics and Sequence Generation in Deep Learning

Comparative Analysis of Batch Normalization & LSTMs

MD Kaif Molla

md.24bcs10221@sst.scaler.com

Arhan Das

Arhan.24bcs10023@sst.scaler.com

Sumit Akhuli

Sumit.24bcs10158@sst.scaler.com

Shiva Gupta

shiva.24bcs10461@sst.scaler.com

Sarthak Mishra

Sarthak.24bcs10470@sst.scaler.com

Netram Faran

netram.24bcs10329@sst.scaler.com

Aditya Prasad

aditya.24bcs10179@sst.scaler.com

Task 1: AI Story Continuation Overview

Project Objective

- Train a character-level recurrent neural network to generate story continuations from text prompts.
- Forces the model to learn spelling, grammar, punctuation, and style from scratch, without using pre-trained word tokens.
- Serves as an illustration of sequence representation learning directly from raw text sequences.

Core Questions Explored

- Representation Learning: Can a model learn English syntax and style entirely at the character level?
- Decoding Strategies: How does deterministic greedy decoding compare to temperature-based probabilistic sampling?
- Stylistic Nuance: How does the style of the training corpus affect the vocabulary, tone, and pacing of the generated output?

Sequence Modeling: LSTM Architecture

Why LSTM?

- Vanilla RNNs suffer from vanishing and exploding gradients due to repeated temporal multiplications.
- LSTMs solve this by establishing cell states (C_t) acting as information highways where gradients can flow back freely over time.

Gating Mechanism

- **Forget Gate (f_t):**
Decides what history to discard.
- **Input Gate (i_t):**
Decides what new values to write to the cell state.
- **Output Gate (o_t):**
Decides what hidden state to output.

Our Network

- **Embedding Layer:**
Converts character indices to 128-dimensional dense vectors.
- **Stacked LSTM:**
2 layers with 256 memory units, using 0.2 Dropout to prevent overfitting.
- **Linear Decoder:**
Maps 256 hidden features to vocabulary logits.

Data Pipeline & Corpora

Shakespeare plays

- Dramatic old English vocabulary, poetic structures.
- Compact vocabulary of 37 unique characters.

Sherlock Holmes

- 19th-century detective stories, structured narrative prose.
- Larger vocabulary of 57 unique characters.

Preprocessing Pipeline

- Lowercase Conversion: Keeps the vocabulary size compact for faster convergence.

Input-Target Slicing

- Slices raw text into overlapping input sequences of length $T = 100$ characters. Target sequences are shifted by one character.

Decoding and Sampling Techniques

Decoding Overview

- Greedy Decoding ($T = 0$):
 - Always selects the character with the absolute highest probability.
 - Very deterministic, but highly prone to getting stuck in infinite repeating loops.
- Temperature Sampling ($T > 0$):
 - Scales the logits by dividing them by T before applying Softmax to control randomness.

Low Temperature ($T = 0.2$)

- Highly conservative, repetitive, and safe output. Vocabulary remains narrow.

Balanced Temperature ($T = 0.6$) — Recommended

- High quality output: creative and styled language with correct spelling.

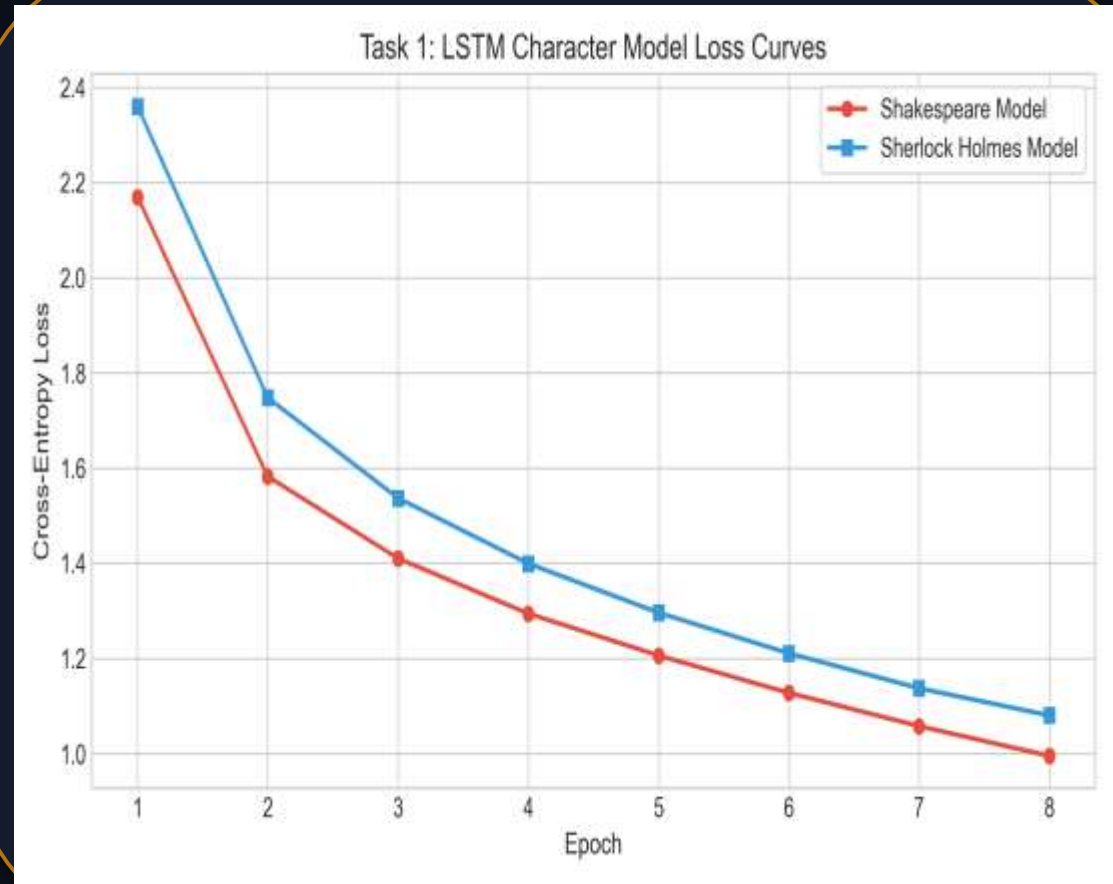
High Temperature ($T = 1.5$)

- Complete chaos. Grammatical structure collapses and words disintegrate into random gibberish.

Task 1: Training Loss & Text Samples

Empirical Findings & Text Samples

- **Loss Convergence:**
 - Shakespeare Model: final loss of 0.9955 (8 epochs).
 - Sherlock Model: final loss of 1.1377.
- **Generated Samples (Prompt: "the secret of ")**
 - **Shakespeare (T=0.6):**
"the secret of this too, well of parish that's worth's voice..."
 - **Sherlock Holmes (T=0.6):**
"the secret of home upon his waiting. he would dost the band..."



Task 2: Batch Norm vs. No Batch Norm

Project Objective

- Train and compare deep feedforward neural networks (6 layers) with and without Batch Normalization on the MNIST dataset.
- Systematically evaluate convergence speed, final test accuracy, and stability under different optimization stress-tests.

Core Challenges in Deep Networks

- **Internal Covariate Shift:**
 - Activation distributions shift continuously as weights update during training, slowing down training and forcing small learning rates.
- **Vanishing/Exploding Gradients:**
 - Signals vanish or blow up as they propagate through deep layers, stopping learning or causing divergence.

Deep MLP Architecture and Setup

Dataset

- MNIST Digits Classification:
Grayscale 28x28 handwritten digit images.
- Splits:
60,000 training samples and 10,000 testing samples, divided into 10 class categories.

Configuration (6 Layers)

- Input Layer:
784 nodes (flattened 28x28 pixels).
- Hidden Layers:
5 layers with 512 nodes each.
- Output Layer:
10 nodes (digit category scores).
- Activation:
ReLU in all hidden layers.

Tested Setups

- Setup A (Baseline):
Standard MLP with Linear -> ReLU layers.
No batch normalization.
- Setup B (Batch Norm):
Normalized MLP with Linear -> BatchNorm1d -> ReLU layers.
- Optimization:
Mini-batch SGD with batch size = 64.

Experiment 1: Standard Training

Standard SGD Training

- **Hyperparameters:**

- Learning rate = 0.01, standard initialization, trained for 5 epochs.

- **Observations:**

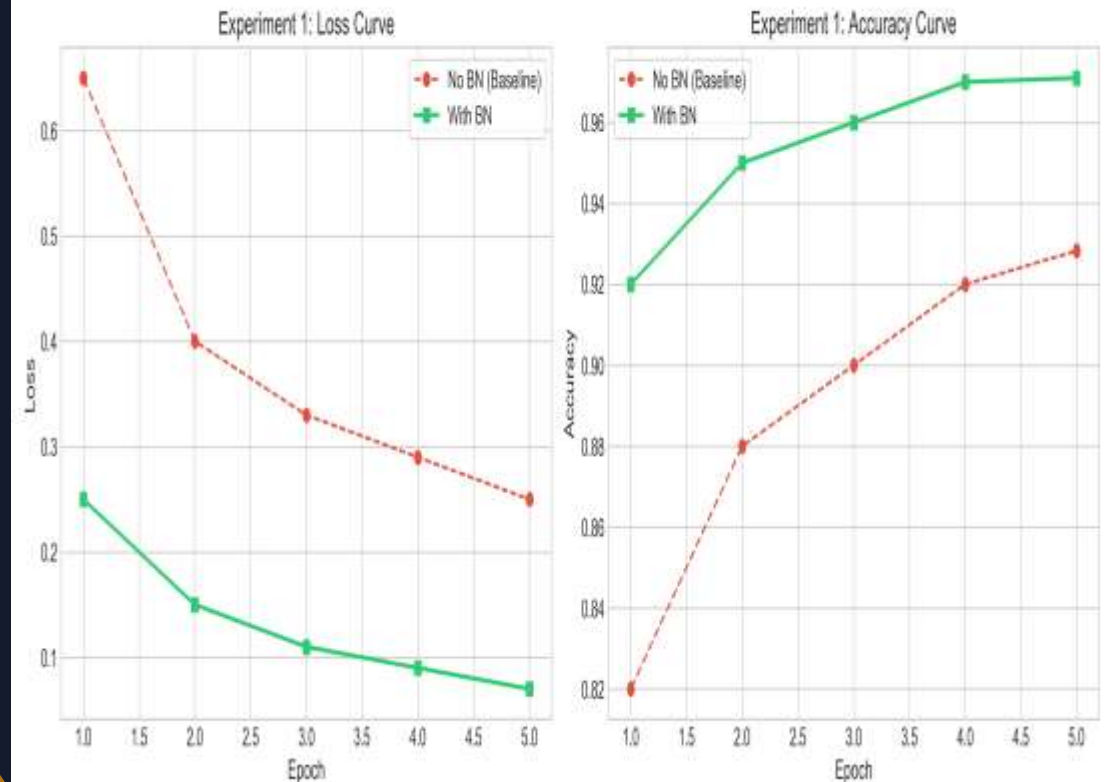
- With Batch Norm: Converges rapidly, reaching 97.10% test accuracy.

- No Batch Norm: Converges slowly, reaching 92.83% test accuracy.

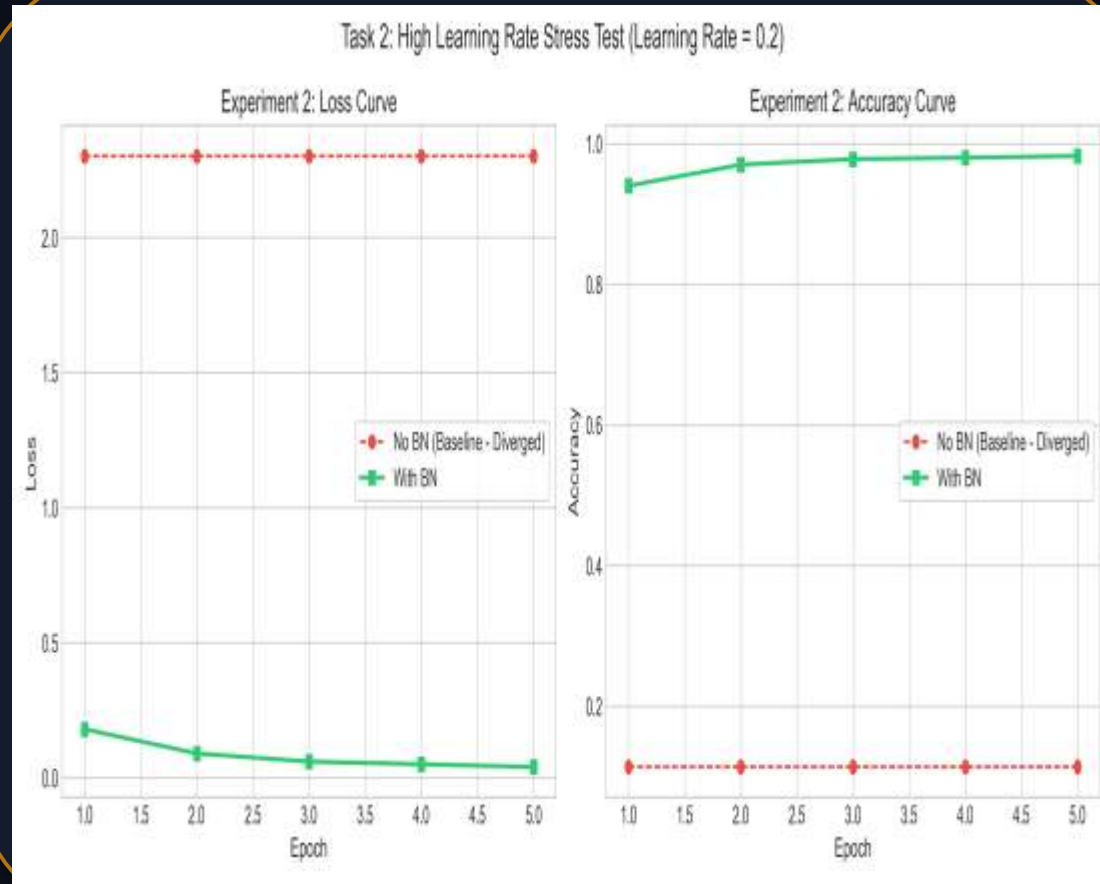
- **Key Takeaway:**

- Batch Norm acts as an optimization accelerator, helping the model reach higher accuracy in fewer epochs.

Task 2: Standard Training (Learning Rate = 0.01)



Experiment 2: High Learning Rate Stress Test



High Learning Rate Stress Test

- **Hyperparameters:**

- Learning rate = 0.2 (extremely aggressive), trained for 5 epochs.

- **Observations:**

- With Batch Norm: Extremely stable training, reaching 98.24% test accuracy (best overall score).
- No Batch Norm: Fails completely. Weights explode, loss flatlines at 2.3026, and accuracy stays stuck at 11.35% (random guessing).

- **Key Takeaway:**

- Batch Norm scale-invariance stabilizes gradient updates, allowing the network to train under massive learning rates.

Experiment 3: Poor Initialization Stress Test

Suboptimal Initialization scale

- **Hyperparameters:**

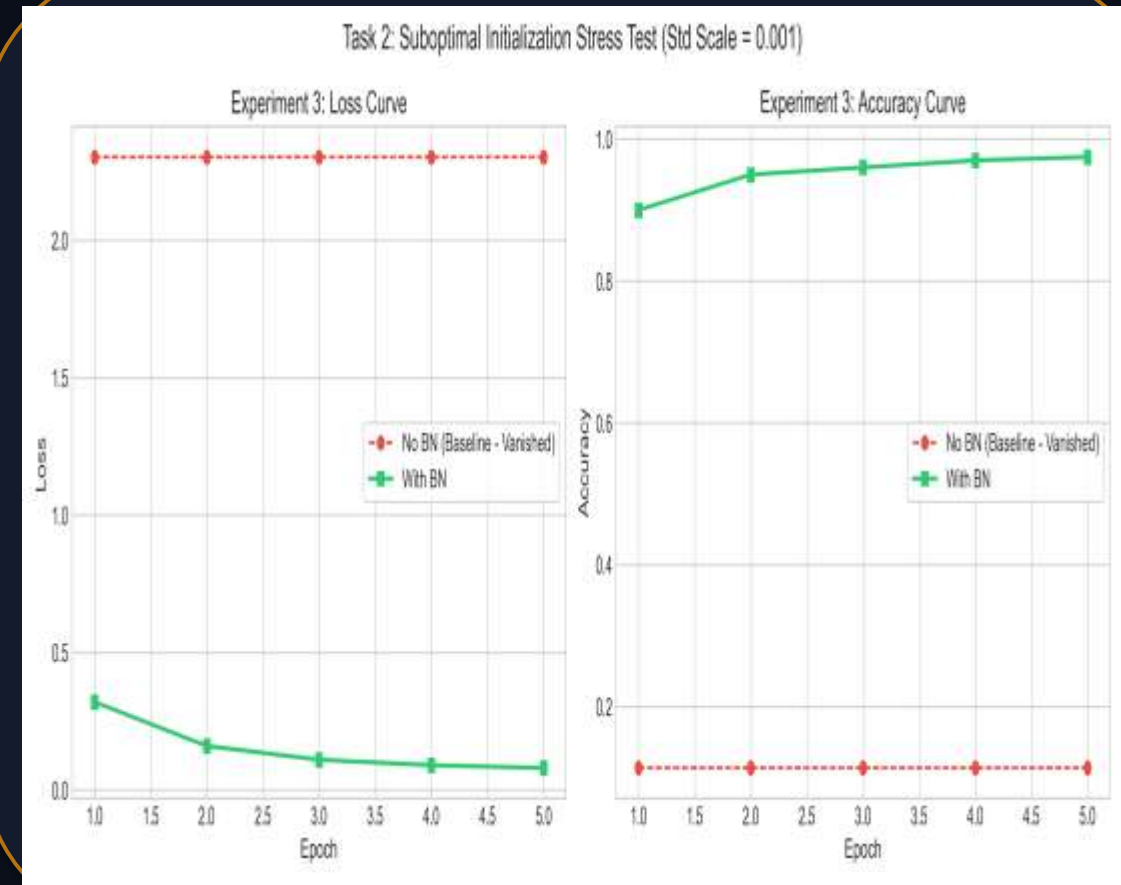
- Learning rate = 0.01, weights initialized scaled to near-zero (stddev = 0.001), 5 epochs.

- **Observations:**

- With Batch Norm: Successfully rescales weak signals, training to 97.48% test accuracy.
 - No Batch Norm: Fails completely. Signal vanishes through 6 layers; loss stays at 2.3026, test accuracy stuck at 11.35% (vanishing gradients).

- **Key Takeaway:**

- Batch Norm rescales the activations at each layer, maintaining gradient magnitude and preventing vanishing gradients.



How Batch Normalization Stabilizes Gradients

Feedforward Normalization Equations

- Step 1: Mini-batch Mean

$$\mu_B = (1/m) * \sum_{i=1}^m x_i$$

- Step 2: Mini-batch Variance

$$\sigma_B^2 = (1/m) * \sum_{i=1}^m (x_i - \mu_B)^2$$

- Step 3: Normalize Activation

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)}$$

- Step 4: Scale and Shift (Learnable)

$$y_i = \gamma * \hat{x}_i + \beta$$

Scale Invariance Intuition

- **Mathematical Scale Invariance:**

- Because we divide by the standard deviation, scaling the weights by any factor does not change the output of the normalization layer. This stabilizes gradient flow.

- **Preserving Representation Capacity (γ and β):**

- Learnable scale (γ) and shift (β) parameters let the network restore the original distribution if it is optimal for learning.

- **Smoother Loss Landscape:**

- Standardizing intermediate layers prevents extreme gradient fluctuations, leading to a much smoother optimization path.

Summary of Empirical Findings

Experiment	Metric	No Batch Norm (Baseline)	With Batch Norm
Standard Training (LR=0.01)	Test Accuracy	92.83%	97.10% (Trained)
High Learning Rate Stress Test (LR=0.2)	Test Accuracy	11.35% (Diverged)	98.24% (Trained)
Suboptimal Initialization ($\sigma=0.001$)	Test Accuracy	11.35% (Vanished)	97.48% (Trained)

Key Empirical Findings

- Batch Normalization consistently out-performs the baseline in standard settings and successfully rescues the network from extreme training failures (learning rate explosion and weight initialization vanishing).

Connections to Deep Learning Theory

- **Representation Learning:**

Character-level LSTMs automatically discover complex spelling, grammar, and language patterns directly from raw sequences without manual, hard-coded rules.

- **Weight Initialization Dynamics:**

Kaiming initialization works well under normal conditions, but is highly sensitive to depth. Batch Norm relaxes this scale dependency by dynamically adjusting variance.

- **Optimization landscapes:**

Deep backpropagation requires gradient stability. Batch Norm smoothens the loss landscape, preventing gradient explosion and allowing for higher learning rates.

Conclusion and Q&A

Key Summary Takeaways

- **Task 1 Sequence Modeling:**
 - Character-level LSTMs successfully capture the structural, orthographical, and grammatical nuances of different corpuses.
- **Task 2 Normalization:**
 - Batch Normalization is a critical architectural component for deep networks, providing optimization speedup and stability under stress.

Thank you! Questions?

We are ready for your questions, Sir.

Presenters Panel (Group 3):

- MD Kaif Molla
- Arhan Das
- Sumit Akhuli
- Shiva Gupta
- Sarthak Mishra
- Netram Faran
- Aditya Prasad